

AIPL におけるメソッド戻り値型の推論

— reply の引数から型は決まるか —

ABCL^c+ / AIOS 処理系

2026 年 7 月 29 日

概要

AIPL (ABCL^c+) の型健全性・型安全性の検討の中で、「メソッドの戻り値型を宣言すべきだ」という提案が出た。本稿はそれに対する「reply の引数から推論できないのか」という問いを出発点とし、(i) 推論は実際に可能であること、(ii) しかし推論だけでは決まらない場面が 6 つあること、(iii) 型健全性の証明にとって本質的なのは推論アルゴリズムではなく宣言という構文であること、を整理する。あわせて推論部分を OCaml 実装 (src/infer.ml, src/types.ml) に実装し、実測した結果を報告する。実装により now / future の戻り値型が any から具体型へ変わり、既存 63 本のコーパスでは 62 本が判定不変、1 本は誤ったエラーが解消した。副次的に、オーバーロード解決 pick_overload に潜んでいた 2 つの欠陥 (失敗候補の単一化が巻き戻らない / 候補の優先順位が登録順の逆) を発見し、これも修正した。

目次

1	問題設定	2
1.1	出発点：実装の現状	2
2	推論できる部分	3
3	推論では決まらないこと	4
3.1	reply しないパスが黙って通る	4
3.2	分岐ごとに異なる型を reply したときの blame	4
3.3	アクター間の相互再帰	4
3.4	リモートアクターと分離コンパイル	4
3.5	become と複数実装	4
3.6	session protocol への昇格	5
4	型健全性の証明にとっての意味	5
5	既存言語はどうしているか	5
6	実装	6
6.1	ρ の表	6
6.2	1 パス目： ρ の確保と defaulting	6

6.3	2 パス目: <code>reply</code> の単相束縛	6
6.4	送信地点	7
7	副産物: <code>pick_overload</code> の 2 つの欠陥	7
8	実験	8
8.1	サンプル	8
8.2	退行試験	8
9	残る限界	9
10	結論と次の一歩	9
付録 A	サンプル全文と実行結果	9
A.1	<code>s1_ok.abcl</code> — <code>reply</code> からの推論	10
A.2	<code>s2_missing_reply.abcl</code> — <code>reply</code> の書き忘れ	11
A.3	<code>s3_conflict.abcl</code> — 分岐ごとに異なる型を <code>reply</code>	11
A.4	<code>s4_chain.abcl</code> — アクターを跨いだ伝播	12
A.5	<code>s5_overload_ambiguity.abcl</code> — <code>principal type</code> が無い例	12
A.6	<code>s6_usable_result.abcl</code> — 通るようになった正しいプログラム	13
付録 B	型検査ドライバ <code>tc_main.ml</code>	14
付録 C	変更ファイル	15
付録 D	参照	15

1 問題設定

AIPL のメソッドは値を「返す」のではなく、`reply(v)` によって呼び出し側の `future` を解決する。

```
class Calc {
  method twice(a) {
    reply(a * 2);
  }
}
var c = new Calc();
var s = now c.twice(21);
```

ここで `s` の型は `int` であってほしい。しかし検討開始時点の型検査器はそう推論していなかった。

1.1 出発点：実装の現状

`src/typing_env.ml` において `reply` は次のように、囲んでいるメソッドと一切結び付かない多相プリミティブとしてグローバル環境に置かれていた。

```
add_poly e "reply" (Forall ([(!a).id], TFun ([TVar a], TUnit)))
```

```
(* reply : forall 'a. 'a -> unit *)
```

そしてメソッドの型は `src/infer.ml` の `preinfer_all_classes` において戻り値 `unit` 固定で構築されていた（コメントにも「いまは戻り値を `unit` としておく」とある）。

```
let tfun = Types.TFun (ps', Types.TUnit) in
```

その結果、送信地点は戻り値型を捨てていた。

```
| FutureSend (target, mname, args) -> ...
  (match repr (Types.instantiate sc) with
   | TFun (param_tys, _ret_ty) ->      (* _ret_ty は使われない *)
     List.iter2 (Types.unify ~loc:e.loc) param_tys arg_tys)
  ...;
  TFuture TAny
| NowSend (target, mname, args) ->
  ignore (infer_expr env { e with desc = FutureSend (...) });
  TAny
```

規則として書けば次のとおりで、`any` が「静的に内容を保証しない境界」として残っていた。

$$\frac{\Gamma(a) = \text{actor}(C) \quad C.m : (\tau_1 \times \dots \times \tau_n) \rightarrow \text{unit} \quad \Gamma \vdash e_i : \tau_i}{\Gamma \vdash \text{now } a.m(\vec{e}) : \text{any}} \text{ NOW-OLD}$$

2 推論できる部分

結論から言えば推論は可能で、機構としては単純である。メソッド $C.m$ ごとに戻り値型変数 $\rho_{C,m}$ を 1 つ確保し、

- 本体中のすべての `reply(v)` 地点で $\rho_{C,m}$ と v の型を単一化する、
- すべての `now / future` 送信地点で $\rho_{C,m}$ を戻り値として返す、

とすればよい。 ρ は union-find の link を通じて伝播するので、本体と呼び出し側のどちらが先に検査されても制約は届く。

$$\frac{\frac{ret(C, m) = \rho \quad \Gamma \vdash e : \rho}{\Gamma \vdash_{C.m} \text{reply}(e) : \text{unit}} \text{ REPLY}}{\Gamma(a) = \text{actor}(C) \quad C.m : (\tau_1 \times \dots \times \tau_n) \rightarrow \rho \quad \Gamma \vdash e_i : \tau_i \quad ret(C, m) = \rho}{\Gamma \vdash \text{future } a.m(\vec{e}) : \text{future } \rho} \text{ FUTURE}$$

$$\frac{\Gamma \vdash \text{future } a.m(\vec{e}) : \text{future } \rho}{\Gamma \vdash \text{now } a.m(\vec{e}) : \rho} \text{ NOW}$$

実装上の要点は 1 つだけある。 ρ をメソッドの型スキームに埋めてはいけない。 $\forall$ に ρ が量化されると `instantiate` が呼び出しごとに ρ を別の変数へ差し替えるため、`reply` 側で付いた制約が呼び出し側へ伝わらなくなる。 ρ は単相のまま別表に持つ。

3 推論では決まらないこと

以下の 6 点は、 ρ を導入しても推論だけでは決着しない。これが「宣言せよ」という提案の実質的な中身である。

3.1 reply しないパスが黙って通る

どこにも `reply` が無い、あるいは一部の分岐でしか `reply` しない場合、 ρ は制約が付かないまま一般化され $\forall \rho. \rho$ となる。呼び出し側は「どんな型の値でも返る」と見なすが、実行時にはその `now` は永久に返らない。型が嘘をつく。

宣言があれば「全パスがちょうど一度、宣言型で `reply` する」という被覆・線形性検査が書けるが、推論は照合先を持たないためこの義務をそもそも述べられない。

3.2 分岐ごとに異なる型を reply したときの blame

`if p reply(1) else reply("no")` は単一化の失敗にはなるが、エラーは「2 番目に検査された `reply` 地点」に出る。数百行離れた分岐のどちらが誤りかは決められない。宣言があれば各 `reply` が独立に照合され、エラー位置が正しくなる。

3.3 アクター間の相互再帰

$A.m$ が `now b.n()` の結果を `reply` し、 $B.n$ が `now a.m()` に依存すると、 $\rho_{A,m}$ と $\rho_{B,n}$ が相互依存する。単相であれば単一化で回るが、多相にした瞬間に polymorphic recursion (決定不能) になる。現在の `preinfer_all_classes` による全プログラム事前走査が成立しているのは戻り値が `unit` 固定だったからで、変数にすると呼び出しグラフ上の不動点計算になる。

3.4 リモートアクターと分離コンパイル

`RemoteTarget` は型検査を丸ごとスキップしている。本体が別ノードにあるため推論のしようがない。 `web_expose` も同様である。AIPL は分散言語であるから、宣言は追加の負担ではなく、リモート境界を型付けする唯一の手段である。推論はソースが手元にある場合しかカバーできず、それは最も安全な場合である。

3.5 become と複数実装

振る舞い置換や、同じメッセージに複数クラスが応答する構成では、メッセージの型は「一つの本体の性質」ではなく「複数本体にまたがる契約」である。独立に推論した型の最小上界は `principal` にならない。

3.6 session protocol への昇格

protocol string を型レベルの session environment へ昇格させるには、 $!m(T).?R.\dots$ の R がインタフェースに書かれていることが前提となる。プロトコルの半分为本体を読まないといけない状態では昇格できない。

4 型健全性の証明にとっての意味

これが提案の核心である。now $a.m(v)$ が飛行中のとき、preservation は構成 (configuration / store) の型付け不変条件として「pending future f は型 T を持つ」を述べる必要がある。

T が「callee の reply 群を単一化した結果」としてしか定義されていないと、不変条件が callee の本体を参照してしまい、subject reduction が合成的でなくなる。callee が一歩簡約するたびに T の意味が変わりうるので、「推論は簡約に対して安定である」という補題が別途必要になる。宣言された T ならば不変条件は $T = \text{ret}(C, m)$ という、簡約で動かない構文片で済む。

注意 1. 推論はアルゴリズムの話、宣言はモデルに現れる構文の話である。証明が要求しているのは後者であり、両者は排他ではない。

5 既存言語はどうしているか

「推論を採用しているか」という観点で 4 つ (+ 1) を調べた結果を表 1 に示す。共通する不変条件は推論の有無に関わらず、返信の型は必ずどこかに人手で書かれていることで、違うのは書く場所だけである。

	本体内の推論	メソッド戻り値	アクター境界
Akka Typed (Scala)	局所推論あり	本体から推論可 (再帰は除く)	<code>ActorRef[R]</code> として宣言
Erlang	Dialyzer の success typing	静的型なし	<code>term()</code> (= any)
Pony	ローカル変数のみ	省略時は推論せず <code>None</code>	behaviour は返信を持たない
Encore	<code>let</code> のみ	明示必須	宣言から <code>Fut[T]</code>
Gleam	HM 系の完全推論	注釈不要	<code>Subject(msg)</code> として宣言

表 1 アクター言語における戻り値型の扱い

とくに 2 点が本稿に直結する。

- **Pony** : 「戻り値型を省略した関数は `None` を返す」。本体から推論するのではなく `unit` へ defaulting する。§3 第 1 項への対処として、本実装もこの方式を採った。
- **Gleam** : 完全な HM 推論を持ちながら、アクターのメッセージ型だけは宣言させる。「推論できる言語ですら境界は書かせる」という最も強い例である。

なお Erlang は `-spec` が強制されない任意注釈であり、`gen_server:call/2` の戻り値は `term()`

である。すなわち「宣言する側の前例」ではなく、AIPL の従来状態 (any) と同じ側に属する。

6 実装

src/types.ml (+41 行) と src/infer.ml に変更を加えた。

6.1 ρ の表

```
(* "Class#method" -> rho. Monomorphic on purpose: if rho were quantified
    in the method scheme, instantiate would replace it at every call site
    and the constraint collected at the reply sites would never reach the
    caller. *)
let method_ret_tys : (string, ty) Hashtbl.t = Hashtbl.create 97
let method_ret_key cls m = cls ^ "#" ^ m
let register_method_ret cls m t = Hashtbl.replace method_ret_tys (method_ret_key cls m) t
let lookup_method_ret cls m = Hashtbl.find_opt method_ret_tys (method_ret_key cls m)
let reset_method_rets () = Hashtbl.reset method_ret_tys
```

6.2 1 パス目： ρ の確保と defaulting

preinfer_all_classes 内の infer_method で ρ を確保し、表に登録する。本体に reply が構文的に現れないメソッドはその場で unit へ落とす。

```
let rho = Types.TVar (Types.fresh_tvar ()) in
Types.register_method_ret c.Ast.cname m.Ast.mname rho;
if not (stmt_has_reply m.Ast.body) then
  Types.unify ~loc:Location.dummy rho Types.TUnit;
(* Keep rho free in the environment so that generalize does not quantify it *)
set_var_scheme env_m "reply"
  (Types.Forall ([], Types.TFun ([rho], Types.TUnit)));
let tfun = Types.TFun (ps', rho) in
```

ρ を env_m の reply に置いているのは、generalize (ftv_env env_m) の自由変数集合に ρ を含めて量化を防ぐためである。

6.3 2 パス目：reply の単相束縛

check_decl のメソッド本体検査環境で、グローバルの $\forall \alpha. \alpha \rightarrow \text{unit}$ を隠すように reply を単相で束縛する。

```
let rho =
  match Types.lookup_method_ret c.Ast.cname m.mname with
  | Some t -> t
  | None -> TVar (Types.fresh_tvar ())
in
set env_m "reply" (Forall ([], TFun ([rho], TUnit)));
check_stmt env_m m.body
```

reply 専用の型付け関数 `check_reply` も設けた。`pick_overload` 任せにすると "no overload of reply matches (string)" という、どの reply と衝突したのか分からない文言になるためである。現在は次のように出る。

```
line 5, col 37: reply type mismatch:
  this method replies with int elsewhere, but with string here
```

6.4 送信地点

`FutureSend` の戻り値を `TFuture TAny` から ρ に変え、`NowSend` は future を剥がして同じ ρ を返す。リモート宛先だけは `any` のままとした (§3 のとおり、ここは宣言でしか埋まらない)。

7 副産物：pick_overload の 2 つの欠陥

ρ を可視化した結果、`method add(a,b) { reply(a+b); }` の推論結果が `(int * int) -> string` という自己矛盾したシグネチャになることが判明した。原因は本変更ではなく、既存の `pick_overload` にあった。

- (1) 失敗候補の単一化が巻き戻らない `List.for_all2` は途中で `false` になると打ち切るため、それまでに結んだ束縛 (例: 第 1 引数 `:= string`) が残ったまま次の候補へ進む。候補を試すたびに引数の型が汚染される。
- (2) 候補の優先順位が登録順の逆 `typing_env` は overload を `sch :: prev` で積むので、リストの先頭は最後に登録されたものになる。`+` では `('a * string) -> string` が先頭であり、引数が両方とも未束縛の型変数のときは必ずこれが最初に一致する。`a + b` が問答無用で `string` に潰れていたのはこれが理由である。

修正は次の 3 点からなる。

1. 各トライアルの前後で型変数の link をスナップショット・復元し、候補試行が副作用を残さないようにする。link は `repr` が `cell := {!cell with link}`、`prune` が `(!cell).link <- ...` と 2 通りの書き換えをするため、レコードを共有したまま保存すると復元にならない。link を値として保存し、新しいレコードを作り直して差し替える。
2. 候補を「引数型が具体的なもの優先 → 引数側の型変数を束縛しないもの優先 → 登録順」で順位付けする。
3. 最上位に戻り値型の異なる候補が複数残る場合は真に曖昧なので報告する。既定では警告、`AIOS_STRICT_OVERLOAD=1` でエラーになる。

```
[warn] line 10, col 13: ambiguous overload of + for ('a, 'a):
  candidate return types are float / int (picked float)
```

注意 2. `a + b` において `a, b` が未束縛のとき、この overload 集合のもとでは principal type が存在しない。`reply` の型が推論だけでは決まりきらないのと同じ現象であり、注釈を要求する動機はここでも共通している。

8 実験

8.1 サンプル

6 本のサンプルで、パッチ前 (git HEAD) と後の判定を比較した。

サンプル	パッチ前	パッチ後
s1 基本	OK	OK。 <code>twice:(int)->int,</code> <code>greet:(string)->string,</code> <code>log:('a')->unit</code>
s4 アクター跨ぎ	OK	OK。 <code>Store#get->int</code> が <code>Front</code> の <code>now</code> を経て <code>Front#fetch->string</code> へ伝播
s6 <code>now</code> の結果を算術に使う	no overload of + matches (any, int)	OK
s3 分岐で異なる型を reply	OK (見逃し)	reply type mismatch: ... int ... string
s2 reply 書き忘れ	(any, int)	(unit, int) (原因が読める文言に)
s5 overload の曖昧性	-> string (黙って)	-> float + ambiguous 警告

表2 サンプルにおけるパッチ前後の判定

s6 が「正しいプログラムが通るようになった」、s3 が「誤ったプログラムが落ちるようになった」であり、両方向に効いている。

8.2 退行試験

`abclc/*.abcl` 63 本について、git HEAD 版と修正版の型検査結果を比較した。

判定が一致	62 / 63
判定が変化	1 / 63
曖昧警告が出たファイル	2

変化した 1 本 `RotateThreeLines.abcl` は、

```
x1 = cx - dx + x; // cx, dx : float, x : method parameter
```

において、旧実装が `('a * string) -> string` を先に選んで `x` を `string` に固定してしまい、line 19: `type mismatch` という誤ったエラーを出していた。修正後は具体的な `(float * float) -> float` が選ばれ、`x` は正しく `float` に束縛されて型検査を通る。すなわち退行ではなく改善である。

曖昧警告が出た 2 本 (`now_future.abcl`, `web_calc.abcl`) はいずれも `method add(a,b) { reply(a+b); }` の形であり、真に `principal type` を持たない箇所である。

9 残る限界

1. 戻り値は単相である。 ρ を全呼び出し地点で共有するため、戻り値に多相性は持たせられない。多相にすると §3 の相互再帰が決定不能側へ落ちる。
2. 本体と署名がなお別世界である。メソッド本体の仮引数は `check_decl` が振る `fresh` 変数であり、`preinfer` が作ったスキームの引数型変数とは繋がっていない。そのため `s5` のように `(int * int) -> float` という、引数側が呼び出し地点由来・戻り値側が本体由来の混成シグネチャが表示されうる。結線は素朴に行うとクラス本体が呼び出しより先に検査される順序のせいで誤検出が増えるため、本変更では触っていない。
3. リモート境界は `any` のままである。これは実装の手抜きではなく、宣言なしには原理的に埋まらない。

10 結論と次の一步

`reply` の引数からメソッドの戻り値型を推論することは可能であり、実装も 200 行程度で済んだ。効果も実測できた。しかしそれは提案を否定するものではない。

推論が届かないのは (i) `reply` を書き忘れたパス、(ii) 分岐間の `blame`、(iii) アクター間の相互再帰、(iv) リモート・分離コンパイル、(v) `become` と複数実装、(vi) `session protocol` への昇格であり、いずれも AIPL が分散アクター言語であることに由来する。そして型健全性の証明が要求しているのは、推論アルゴリズムではなく簡約で動かない構文としての宣言である。

したがって次の一步は、両者を排他ではなく役割分担として実装することである。

1. AST に省略可能な `method m(x) : T` を追加する。**remote / expose するメソッドは必須** とする。
2. 注釈がある場合：本体環境に `reply : T → unit` を束縛し、全パス被覆検査（ちょうど一度 `reply` する）を行い、送信地点で `T` を使う。
3. 注釈が無い場合：本稿の推論を走らせる。ただし制約の付かない ρ は一般化せず `unit` へ defaulting する (Pony 方式)。ここだけは一般化してはならない。`send` (fire-and-forget) と `now` (ask) の区別がこの一点に乗る。
4. 推論結果のシグネチャを出力し、利用者が貼り付けられるようにする。

いずれの設計でもクラスのメソッド表には結局インタフェースが生まれる。論点は**存在するかどうか**ではなく**誰が書くか**であり、リモートと証明の都合から「人間が書く」側に倒すのが妥当である。その先に、`protocol string` を型レベルの `session environment` へ昇格させる道が開ける。

付録 A サンプル全文と実行結果

本節のサンプルは `docs/samples/reply_inference/` に置いてある。型検査だけを走らせる小さなドライバ `tc_main.ml` (§ 付録 B) を用いて実行した。

```
$ cd src && make thread_repl
```

```
$ ocamlfind ocamlc -package unix -thread -linkpkg -w -a -I src -o tc \
  src/location.cmo src/types.cmo src/ast.cmo src/typing_env.cmo \
  src/infer.cmo src/typecheck.cmo src/parser.cmo src/lexer.cmo \
  docs/samples/reply_inference/tc_main.ml
$ ./tc docs/samples/reply_inference/s1_ok.abcl
```

A.1 s1_ok.abcl — reply からの推論

```
// (1) reply から戻り値型が推論されること
class Calc {
  // reply が int
  method twice(a) {
    reply(a * 2);
  }
}

class Greeter {
  // reply が string
  method greet(name) {
    reply("hello, " + name);
  }
  // reply が無い = tell 型のメソッド。unit へ defaulting されるべき
  method log(msg) {
    print(msg);
  }
}

var calc = new Calc();
var g = new Greeter();

var s = now calc.twice(21);
print("twice = " + s);

var f = future g.greet("AIPL");
var msg = await f;
print(msg);
```

```
[OK] type check passed
[method return types (inferred from reply)]
  Calc#twice -> int
  Greeter#greet -> string
  Greeter#log -> unit
[class_method_schemes]
class Greeter
  greet : (string) -> string
  log : ('a) -> unit
class Calc
  twice : (int) -> int
```

`twice` は `int`、`greet` は `string` と推論され、`reply` を持たない `log` は `unit` へ defaulting されている。

A.2 s2_missing_reply.abcl — `reply` の書き忘れ

```
// (2) reply を書き忘れたメソッドの結果を now で使う
// 従来: now の型が any なので素通り → 実行時に永久に返らない
// 今回: bump の戻り値が unit に defaulting され、静的エラーになる
class Counter {
  method bump(n) {
    print("bumped");
  }
}

var c = new Counter();
var x = now c.bump(1);
print(x + 1);
```

```
[Type error] line 12, col 9: no overload of + matches (unit, int)
[method return types (inferred from reply)]
Counter#bump -> unit
```

パッチ前は `(any, int)` というエラーで、「そもそも `bump` が値を返さない」という原因が読み取れなかった。現在は `unit` と出るので、`reply` の欠落が直ちに分かる。

A.3 s3_conflict.abcl — 分岐ごとに異なる型を `reply`

```
// (3) 分岐ごとに違う型を reply する
// ρ が2つの reply 地点を結ぶので、2つ目の reply で衝突が検出される
class Router {
  method route(k) {
    if (k > 0) { reply(1); } else { reply("negative"); }
  }
}

var r = new Router();
var v = now r.route(5);
```

```
[Type error] line 5, col 37: reply type mismatch:
  this method replies with int elsewhere, but with string here
[method return types (inferred from reply)]
Router#route -> int
```

パッチ前はこのプログラムが型検査を通過していた。 ρ が 2 つの `reply` 地点を結ぶことで検出できるようになった。ただし §3 のとおり、エラーが指すのは「2 番目に検査された `reply`」であって、どちらが誤りかは決められない。

A.4 s4_chain.abcl — アクターを跨いだ伝播

```
// (4) アクターをまたいで ρ が伝播すること
//   Store.get の reply -> Front の now の型 -> Front.fetch の reply
class Store {
  method get(k) {
    reply(k * 100);
  }
}

class Front {
  method fetch(k) {
    var v = now store.get(k);
    reply("value=" + v);
  }
}

var store = new Store();
var front = new Front();

var out = now front.fetch(3);
print(out);
```

```
[OK] type check passed
[method return types (inferred from reply)]
  Front#fetch -> string
  Store#get -> int
[class_method_schemes]
class Store
  get : ('a) -> int
class Front
  fetch : (int) -> string
```

Store.get の reply から得た int が、Front.fetch 内の now の型となり、さらに Front.fetch 自身の reply を経て string に至る。ρ が union-find 経由でアクター境界を越えて伝播していることが確認できる。

A.5 s5_overload_ambiguity.abcl — principal type が無い例

```
// (5) 本パッチが顕在化させた、pick_overload の既存の問題
//   a も b も未束縛の型変数なので、+ の4つの overload すべてが「一致」する。
//   pick_overload は先頭の候補を採るが、typing_env は overload を
//   prepend で積むため、先頭は最後に登録された ('a * string) -> string。
//   結果、a + b は string に解決され、b は string に固定されてしまう。
//   (パッチ以前からこの束縛は起きていたが、戻り値が unit 固定だったので
//   シグネチャに現れず、見えていなかった)
class Calc {
```

```

method add(a, b) {
  reply(a + b);
}
}

var calc = new Calc();
var s = now calc.add(1, 2);

```

```

[warn] line 10, col 13: ambiguous overload of + for ('a', 'a'):
      candidate return types are float / int (picked float)
[OK] type check passed
[method return types (inferred from reply)]
  Calc#add -> float
[class_method_schemes]
class Calc
  add : (int * int) -> float

```

AIOS_STRICT_OVERLOAD=1 を与えると警告ではなくエラーになる。

```

TYPE_ERROR: line 10, col 13: ambiguous overload of + for ('a', 'a'):
      candidate return types are float / int

```

表示されている (int * int) -> float は、引数側が呼び出し地点由来、戻り値側が本体由来の混成である (§9 の限界 2)。

A.6 s6_usable_result.abcl — 通るようになった正しいプログラム

```

// (6) 正しいプログラムが通るようになる例
//   パッチ前: now の結果は any なので、算術に使った時点で
//             "no overload of + matches (any, int)" で落ちていた。
//   パッチ後: reply(a * 2) から int と分かるので通る。
class Calc {
  method twice(a) {
    reply(a * 2);
  }
}

var c = new Calc();
var s = now c.twice(21);
var t = s + 1;
print("t = " + t);

```

```

[OK] type check passed
[method return types (inferred from reply)]
  Calc#twice -> int
[class_method_schemes]
class Calc
  twice : (int) -> int

```

パッチ前は line 13: no overload of + matches (any, int) で落ちていた。now の結果が any である限り、算術にも比較にも使えなかったためである。

付録 B 型検査ドライバ tc_main.ml

REPL (SDL に依存する) を起動せずに型検査だけを走らせるための最小ドライバ。

```
(* tc_main.ml — AIPL/ABCL の型検査だけを走らせる小さなドライバ *)

let parse_file (fname : string) : Ast.program =
  let ic = open_in fname in
  let len = in_channel_length ic in
  let s = really_input_string ic len in
  close_in ic;
  let lb = Lexing.from_string s in
  lb.Lexing.lex_curr_p <- { lb.Lexing.lex_curr_p with Lexing.pos_fname = fname };
  Parser.program Lexer.token lb

let () =
  let fname = Sys.argv.(1) in
  Printf.printf "=== %s ===\n%!" (Filename.basename fname);
  match (try Ok (parse_file fname) with e -> Error (Printexc.to_string e)) with
  | Error m -> Printf.printf "[Parse error] %s\n%!" m; exit 2
  | Ok prog ->
    (* クラスメソッドのスキーム表は出さず、戻り値型の表だけ見たいので
       Infer.verbose は false にして、自前に表示する *)
    Infer.verbose := false;
    (match Infer.check_program prog with
    | Ok _ ->
      print_endline "[OK] type check passed";
      Types.debug_print_method_rets ();
      print_endline "[class_method_schemes]";
      Hashtbl.iter
        (fun cls sigs ->
          Printf.printf "class %s\n" cls;
          List.iter
            (fun (m, sch) ->
              Printf.printf "  %s : %s\n" m
                (Types.string_of_ty_pretty (Types.repr (Types.instantiate sch))))
              sigs)
        Types.class_method_schemes
    | Error msg ->
      Printf.printf "[Type error] %s\n" msg;
      Types.debug_print_method_rets ();
    print_newline ())
```

付録 C 変更ファイル

ファイル	変更	内容
src/types.ml	+41	ρ 表、debug_print_method_rets
src/infer.ml	+176 / -29	ρ の確保・伝播、check_reply、pick_overload の修正

付録 D 参照

- Pony Tutorial, *Methods*. <https://tutorial.ponylang.io/expressions/methods.html>
- ponylang/rfcs #0010, *Generic Type Inference*. <https://github.com/ponylang/rfcs/blob/main/text/0010-generic-type-inference.md>
- Fernandez-Reyes et al., *An Optimised Flow for Futures: From Theory to Practice*. <https://arxiv.org/pdf/2107.07298>
- de Boer et al., *Forward to a Promising Future*. https://link.springer.com/chapter/10.1007/978-3-319-92408-3_7
- 本リポジトリ docs/ABCLCP_TYPE_SOUNDNESS_REPORT.md (§5, §12)